



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХТ

Лекция 25. Управление конфигурацией

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Системы управления конфигурацией
-  Введение в Ansible
-  Введение в Salt
-  Infrastructure as Code (IaC)

Системы управления конфигурацией

-  Общий принцип решения всех административных задач
-  Общий репозиторий (хранение в git) → совместная работа
-  Повторяемость
-  Декларативность

Ключевые CM системы

Ansible

- ⦿ Написан на Python
- ⦿ Конфигурация в YAML



ANSIBLE

SaltStack

- ⦿ Написан на Python
- ⦿ Конфигурация в YAML+Python
- ⦿ Транспорт ZeroMQ

SALTSTACK.

Chef

- ⦿ Написан и конфигурация на Ruby

 Progress® Chef®

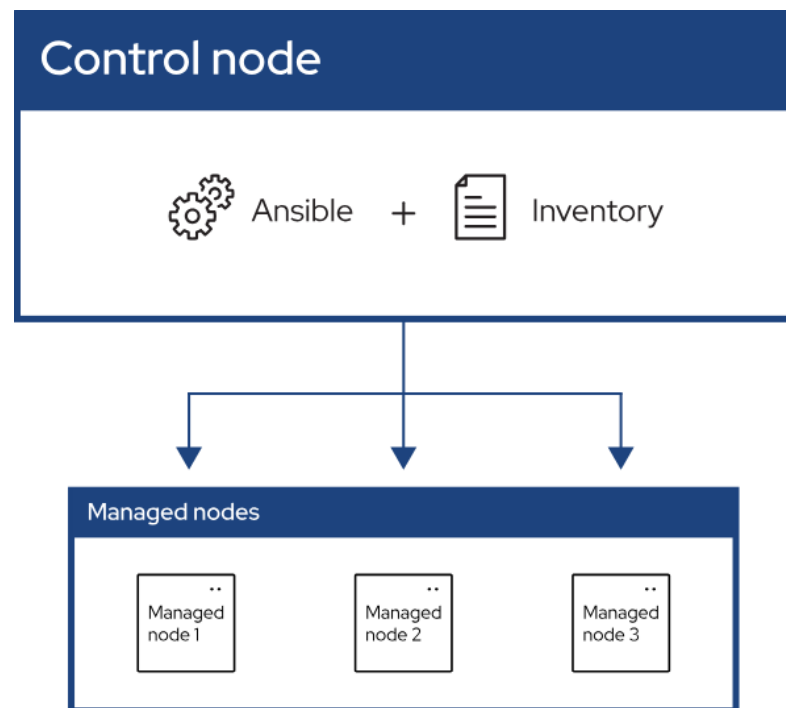
Puppet

- ⦿ Написан и конфигурация на Ruby

perforce
 **Puppet**

Ansible

- Без агента
 - Нужен только ssh
- Низкий порог входа
- YAML
- Много готовых модулей и ролей
 - Ansible Galaxy
- Идемпотентность (запуская много раз, результат одинаковый)
- Поддержка Windows-клиентов



Файлы Ansible

-  `ansible.cfg` — главный конфигурационный файл Ansible
-  `hosts` — файл inventory (с группами хостов)
-  `group_vars/webserver` — директория с описанием переменных для группы хостов
-  `host_vars/database-prod.ru` — переменные для определенного хоста

Пример inventory (hosts файл)

```
[database-prod]  
database1-prod.ru  
database2-prod.ru
```

```
[database-test]  
database-test1.ru
```

```
[webservers]  
apache1-prod.ru  
apache2-prod.ru
```

Пример ansible.cfg

```
[defaults]
inventory = ./hosts
remote_user = ansible
host_key_checking = False
```

```
[privilege_escalation]
become = True
become_method = sudo
become_user = root
become_ask_pass = False
```


Пример playbook

```
- hosts: webservers
  vars:
    welcome_message: "Hello from ansible"
  tasks:
    - name: ensure apache is at the latest version
      yum:
        name: httpd
        state: latest
    - name: setup apache index.html
      copy:
        dest: /var/www/html/index.html
        content: "{{ welcome_message }}"
- hosts: databases
  tasks:
    - name: ensure mariadb is at the latest version
      yum:
        name: mariadb-server
        state: latest
```

Playbook. Plays

```
- hosts: webservers
  vars:
    welcome_message: "Hello from ansible"
  tasks:
    - name: ensure apache is at the latest version
      yum:
        name: httpd
        state: latest
    - name: setup apache index.html
      copy:
        dest: /var/www/html/index.html
        content: "{{ welcome_message }}"

- hosts: databases
  tasks:
    - name: ensure mariadb is at the latest version
      yum:
        name: mariadb-server
        state: latest
```

Playbook. Hosts

```
- hosts: webservers
```

```
vars:
```

```
  welcome_message: "Hello from ansible"
```

```
tasks:
```

```
- name: ensure apache is at the latest version
```

```
  yum:
```

```
    name: httpd
```

```
    state: latest
```

```
- name: setup apache index.html
```

```
  copy:
```

```
    dest: /var/www/html/index.html
```

```
    content: "{{ welcome_message }}"
```

```
- hosts: databases
```

```
tasks:
```

```
- name: ensure mariadb is at the latest version
```

```
  yum:
```

```
    name: mariadb-server
```

```
    state: latest
```

Playbook. Tasks

```
- hosts: webservers
  vars:
    welcome message: "Hello from ansible"
  tasks:
    - name: ensure apache is at the latest version
      yum:
        name: httpd
        state: latest
    - name: setup apache index.html
      copy:
        dest: /var/www/html/index.html
        content: "{{ welcome_message }}"
- hosts: databases
  tasks:
    - name: ensure mariadb is at the latest version
      yum:
        name: mariadb-server
        state: latest
```

Playbook. Modules

```
- hosts: webservers
  vars:
    welcome_message: "Hello from ansible"
  tasks:
    - name: ensure apache is at the latest version
      yum:
        name: httpd
        state: latest
    - name: setup apache index.html
      copy:
        dest: /var/www/html/index.html
        content: "{{ welcome_message }}"
- hosts: databases
  tasks:
    - name: ensure mariadb is at the latest version
      yum:
        name: mariadb-server
        state: latest
```

Playbook. Vars

```
- hosts: webservers
  vars:
    welcome message: "Hello from ansible"
  tasks:
    - name: ensure apache is at the latest version
      yum:
        name: httpd
        state: latest
    - name: setup apache index.html
      copy:
        dest: /var/www/html/index.html
        content: "{{ welcome_message }}"
- hosts: databases
  tasks:
    - name: ensure mariadb is at the latest version
      yum:
        name: mariadb-server
        state: latest
```

Ansible для развертывания систем

- 📡 Можно обновлять кодовую базу на хостах
- 📡 Настраивать файлы конфигурации приложений с помощью templates (jinja2)
- 📡 Хранить файлы с секретами в репозитории в зашифрованном виде (ansible-vault). При запуске playbook эти файлы можно расшифровать и положить на целевой хост
- 📡 Перезапуск сервисов
- 📡 Для контейнеризированных приложений есть модуль docker

Infrastructure as Code (IaC)

 В одном репозитории можно хранить и версионировать:

- ⦿ Файл с требованиями к версии ansible и другим библиотекам (requirements.txt)
- ⦿ Файлы с описанием хостов или групп хостов, в том числе соответствующие переменные
- ⦿ Компоненты, приводящие хосты к определённому состоянию (playbooks или roles, в зависимости от организации репозитория)
- ⦿ Описание компонентов и примеры запуска

Инфраструктура как код (IaC)

- ☯ Применение практик сопровождения разработки ПО в процессах управления конфигурациями:
 - ☉ Объекты или цели конфигурации описываются как наборы свойств (чаще в файлах на git вместо записей в CMDB) и используются как параметры для исполняемых скриптов
 - ☉ Простой, но управляемый процесс сопровождения изменений
 - ☉ Ориентация на тиражирование и масштабирование
- ☯ Оптимально для развертывания исполнительных сред в DevOps с применением виртуализации и контейнеризации. Совместимо с процессами IaaS для ЦОД и управлением облачными структурами.
- ☯ Декларативный, процедурный или интеллектуальный подход к оформлению (что делаем/как делаем/зачем делаем)
- ☯ Push и pull методы доставки конфигурации
- ☯ Управляемое качество + Скорость + Низкая цена

Философия Salt

Скорость и масштабируемость

ZeroMQ — очень быстрая асинхронная библиотека обмена сообщениями («мультисокеты на стероидах», там где реальный MQ избыточен)

Гибкая сетевая топология

Миньон-Мастер, только Миньон, только Мастер, несколько Мастеров

Безопасность

Шифрование везде

Самодостаточность, совместимость и расширяемость

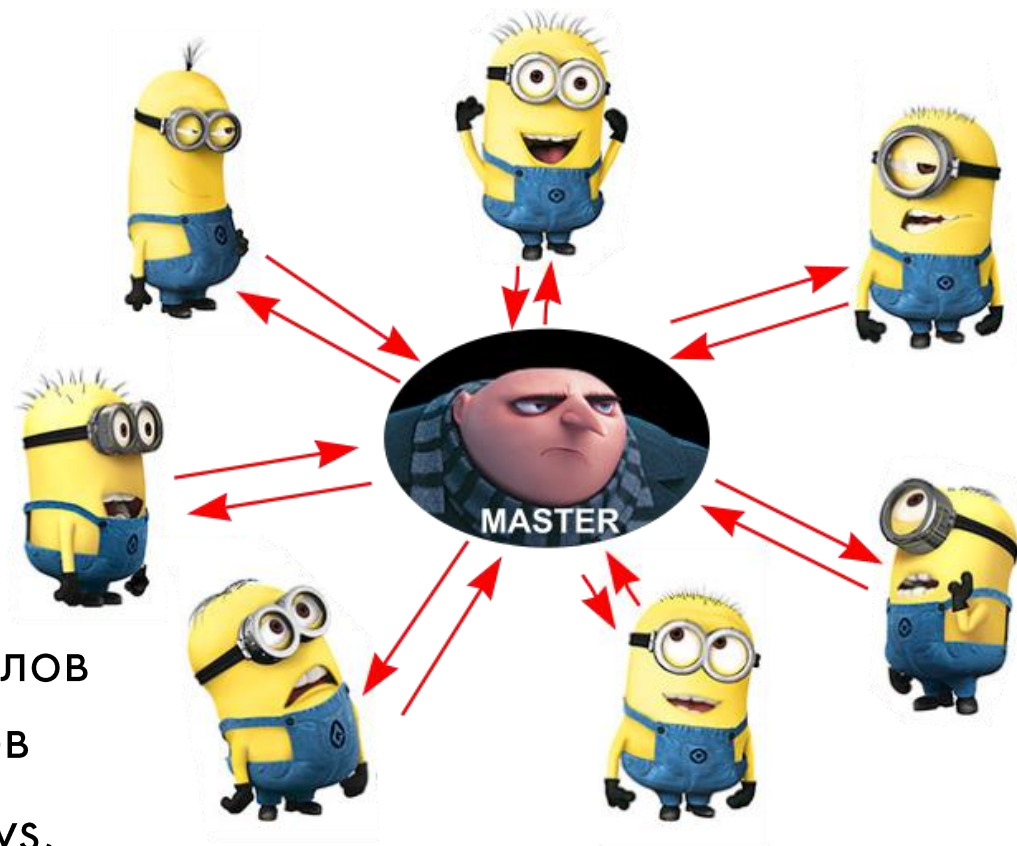
Не требуются дополнительные компоненты, открытый API упрощает интеграцию и создание дополнительных модулей

Детерминированный порядок выполнения

Всякий раз, когда вы запускаете что-то, оно должно выполняться в том же порядке и производить точно такой же результат, независимо от того, сколько раз вы запускаете его, независимо от того, когда вы запускаете его.

Философия Salt

- Параллельная работа миньонов
- До 10^5 миньонов на один мастер
- У миньона может быть несколько мастеров
- Python API/REST API
- Любой тип языка для файлов конфигурации и шаблонов
- Поддержка Linux, Windows, AWS, Azure, Digital Ocean



Что может агент SaltStack



- 📶 Удаленно выполнять команды
- 📶 Удаленно применять изменение конфигурации
- 📶 Собирать и передавать данные о состоянии системы
- 📶 Использовать данные о состоянии других систем
- 📶 Реагировать на изменение состояния

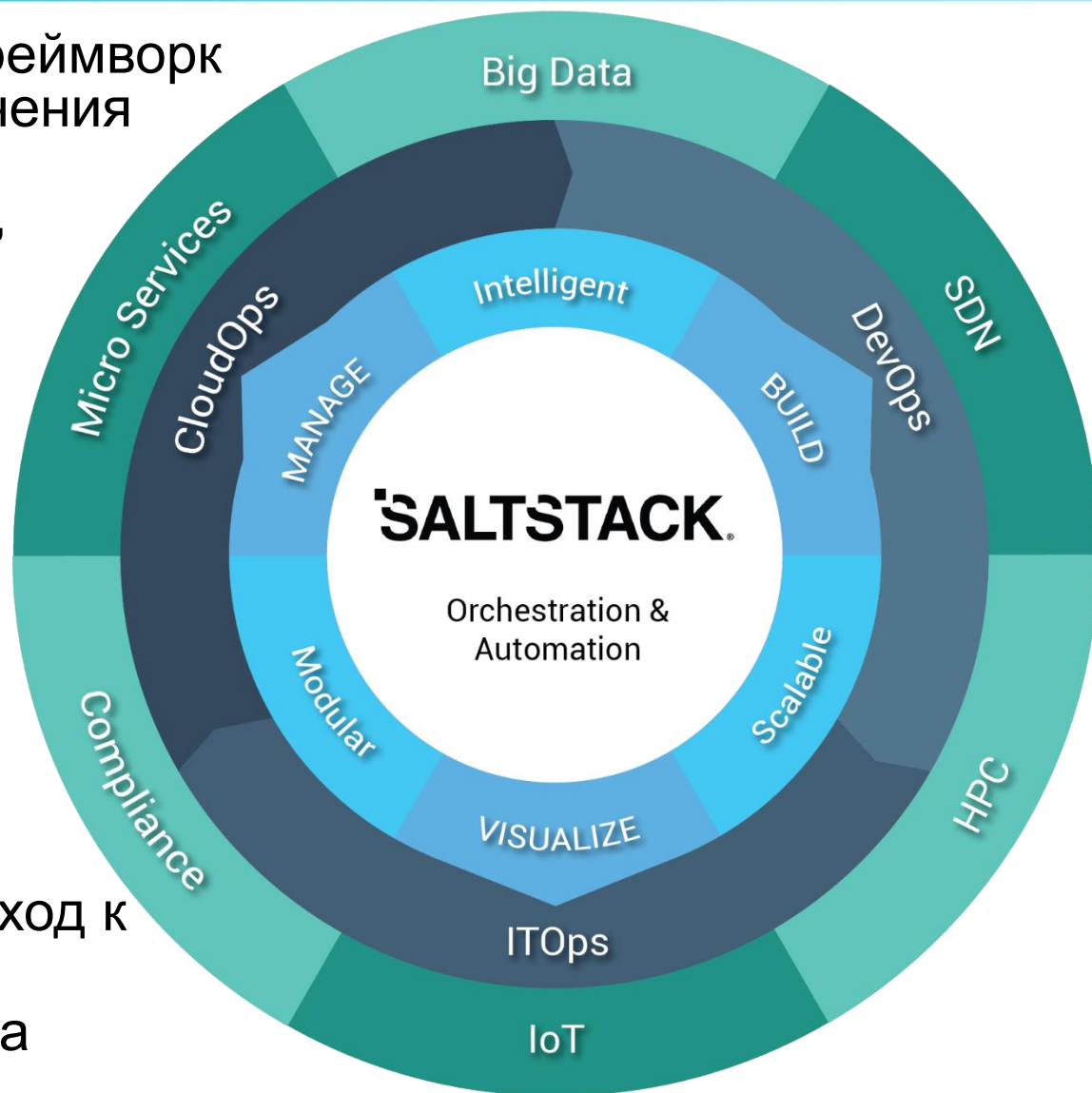
Особенности Salt

- 📶 Может потребоваться установка агента
- 📶 Запущенный процесс не прервать
- 📶 Откат к состоянию до изменения штатно не предусмотрен
- 📶 Порог вхождения (sysop, Python, Jinja)
- 📶 Процесс отладки
- 📶 Совместимость версий
- 📶 и т.д.



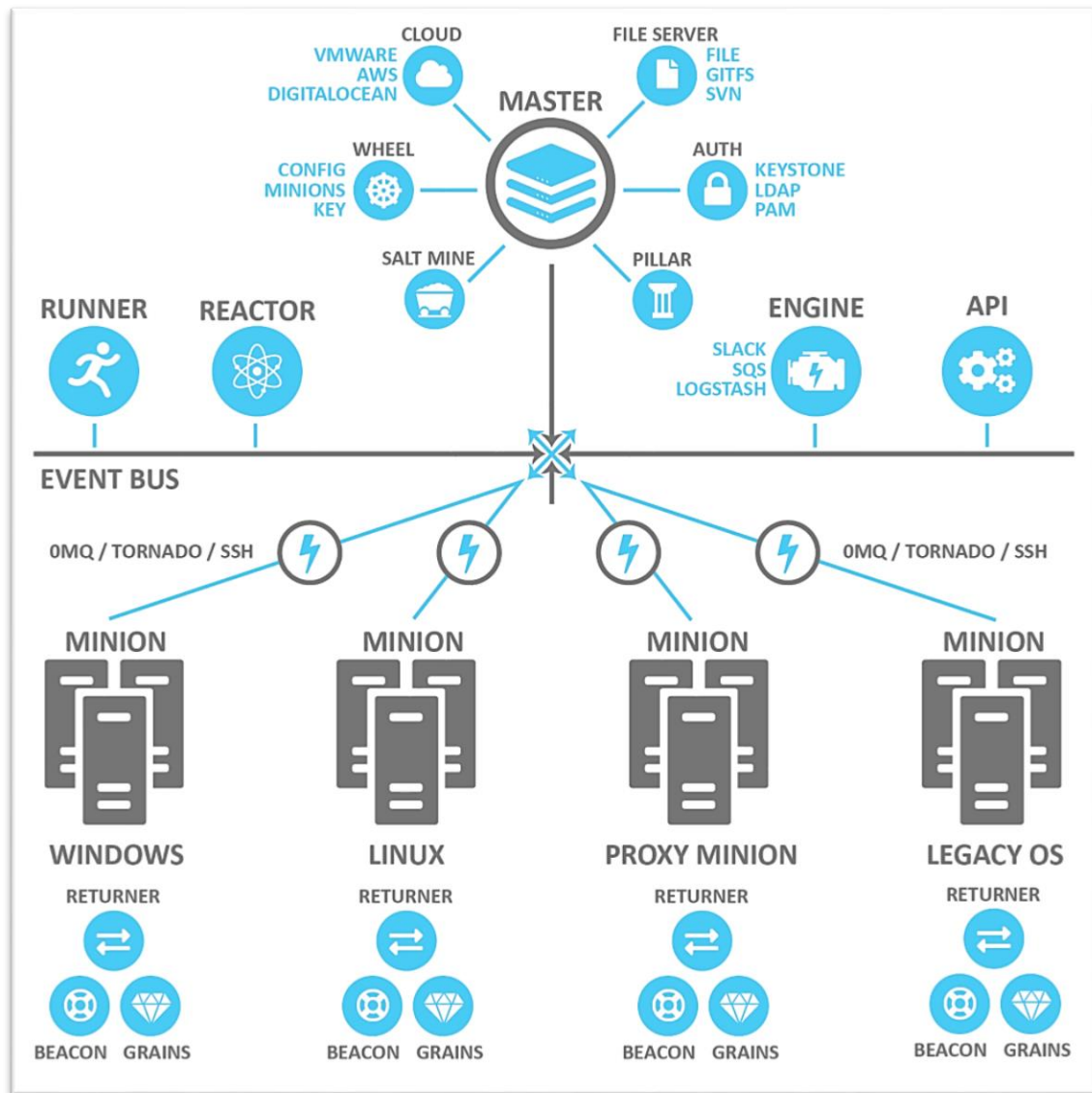
Особенности Salt

- ❶ Универсальный фреймворк удаленного выполнения команд для конфигурирования, автоматизации, настройки и оркестрации
- ❷ Основан на Python
- ❸ Open-source
- ❹ Поддержка IaC для развертывания и управления ЦОД
- ❺ Декларативный и императивный (процедурный) подход к конфигурации
- ❻ Push и pull доставка



Концепция и компоненты

- Master
- Minion
- Event Bus
- Executions
- Formulas
- Grains
- Pillar
- Top files
- Runner
- Returner
- Reactor
- Beacon
- SaltSSH
- Salt Cloud
- Salt Mine
- Wheel
- Engine



Когда Salt наиболее эффективен

-  Настроенные бизнес-процессы IT обслуживания
-  Стандартизированные конфигурации
-  ЦОД SM: тысячи хостов, сотни и тысячи заявок на обслуживание
-  DevOps: сотни исполняемых сред, частые фиксированные изменения
-  Необходимость в оркестрации при запуске сред
-  Гетерогенные среды

Итоги

В данной лекции были рассмотрены следующие темы:

-  Системы управления конфигурацией
-  Введение в Ansible
-  Введение в Salt
-  Infrastructure as Code (IaC)